GCW-ICP: Geometric Confidence-Weighted ICP

Reference implementation for the paper **Geometric Confidence-Weighted ICP for Robust Local Point Cloud Registration Against Outliers**.

GCW-ICP is a robust point-to-plane iterative closest point (ICP) method for local point-cloud registration. It combines a Welsch residual-magnitude weight with a local residual-consistency weight computed from fixed source neighborhoods. The implementation includes adaptive robust scales, an annealed magnitude floor, low-support and rotational-degeneracy safeguards, and an optional coarse-to-fine fallback.

This repository is currently private while the manuscript is under review. The release will be finalized after the review process.

Repository contents

```
.
|-- gcw_icp_p2l.py             # GCW-ICP solver and PLY I/O utilities
|-- run_gcw_icp_local.py       # Registration and Open3D visualization script
|-- benchmark_gcw_icp.py       # Paper-format metric calculation script
|-- source_overlap_20pct.ply   # Example source cloud with 20% overlap
|-- target_overlap_20pct.ply   # Example target cloud with 20% overlap
|-- T_gt_overlap_20pct.npy     # 4x4 ground-truth transform for the overlap
example
|-- source_fair_70pct.ply      # Example source cloud with 70% added outliers
|-- target_fair_clean.ply      # Corresponding clean target cloud
`-- Armadillo.ply              # Additional example point cloud
```

The included point clouds are compact demonstration data for running the code. They are not a release of every dataset, trial record, baseline implementation, or result reported in the manuscript.

Requirements

- Python 3.10 or later
- NumPy
- SciPy
- Open3D (required only for interactive visualization and preferred point-cloud I/O)

Install the dependencies in a virtual environment:

```
python -m venv .venv
.venv\\Scripts\\activate  # Windows PowerShell
python -m pip install --upgrade pip
python -m pip install numpy scipy open3d
```

Without Open3D, the registration core can still read standard ASCII or binary little-endian vertex-only PLY files and estimate normals with SciPy. The visualization script requires Open3D.

Quick start

Run GCW-ICP on the provided partial-overlap example:

```
python run_gcw_icp_local.py \
  --target target_overlap_20pct.ply \
  --source source_overlap_20pct.ply \
  --ground-truth T_gt_overlap_20pct.npy \
  --output registered_source_overlap_20pct.ply
```

The script saves the registered source cloud, prints the final optimization diagnostics and paper-format error metrics, then opens before- and after-registration visualizations. Close each window to continue.

To run without the optional coarse-to-fine fallback, provide an externally obtained initial transform:

```
python run_gcw_icp_local.py \
  --target target_overlap_20pct.ply \
```

```
  --source source_overlap_20pct.ply \
  --init-transform initial_transform.npy \
  --output registered_source_overlap_20pct.ply
```

initial_transform.npy must contain a valid 4×4 initial pose estimate. The provided ground-truth transform is intended only for evaluation and should not be used as an initialization in a benchmark comparison.

For a forced coarse-to-fine run, use --multiscale-mode always.

Benchmark metrics

benchmark_gcw_icp.py computes the metrics used in the manuscript:

- rotation error (degrees)
- translation error
- normalized translation error
- normalized transformation RMSE
- binary success, defined as rotation error below 1 degree and normalized transformation RMSE below 1e-2

Use it with a source cloud, target cloud, and the corresponding 4x4 ground-truth transformation:

```
python benchmark_gcw_icp.py \
  --target target_overlap_20pct.ply \
  --source source_overlap_20pct.ply \
  --ground-truth T_gt_overlap_20pct.npy \
  --output-json gcw_icp_benchmark.json
```

All paths in the examples are relative to the repository root. The scripts also accept absolute paths to user-provided PLY files and NumPy .npy transformation files.

Input and output conventions

- Point clouds must contain Cartesian x, y, and z coordinates. Normals (nx, ny, nz) are used when present and otherwise estimated.
- A ground-truth transform is a 4 x 4 NumPy array stored in .npy format.
- The solver uses the row-vector convention: registered = source @ R.T + t.
- The output point cloud is the transformed source cloud in PLY format.

Main options

Option	Default	Meaning
--max-iter	60	Maximum iterations at each resolution level
--min-iterations	30	Minimum iterations before convergence is accepted
--k-nn	10	Fixed source-neighborhood size for local consistency
--tol	1e-6	Rotation and translation update tolerance
--damping	1e-8	Point-to-plane normal-equation damping
--multiscale-mode	fallback	fallback evaluates the hierarchy only when triggered; always forces it
--fallback-distance	0.01	Normalized median correspondence distance that triggers the fallback

Reproducibility notes

The controlled manuscript comparisons use fixed perturbation settings: a 30-degree initial rotation, translation equal to 2% of the model diagonal, independent 80% subsampling, and Gaussian noise with standard deviation 0.1% of the model diagonal. Outlier and partial-overlap conditions are evaluated separately. The solver is a local refinement method and does not replace feature-based or globally certifiable initialization.

Data and code availability

Before making this repository public, verify that every bundled point cloud and any later-added experimental data can be redistributed under its original license or the relevant data agreement. Do not upload raw scans, proprietary workpiece data, or third-party benchmark files unless redistribution is permitted.

Citation

Citation information will be added when the manuscript is accepted or published.

License

No license has been selected yet. Until a license is added, the code and data are not granted for reuse, redistribution, or modification.